

컴퓨터비전 _인공지능 (AI) 개론

3. 파이토치의 기본 기능

- 텐서
- 자동 미분 기능
- 2차 함수의 경사 차원산
- 시그모이드 함수의 경사 차원산

딥러닝 모델을 만든다는 것은 결국 숫자로 이루어진 데이터를 변환하고 차원산하는 과정입니다. 고객 정보, 이미지 픽셀, 문장의 단어 등 어떤 종류의 데이터든 모델이 이해하려면 표준화된 숫자 컨테이너에 담아야 합니다.

PyTorch에서는 이 컨테이너의 역할을 하는 것이 바로 **텐서(Tensor)**입니다. 텐서는 단순한 숫자 배열을 넘어, 딥러닝의 핵심 기능인 자동 미분과 GPU 가속을 지원하도록 설차원된 특별한 데이터 구조입니다.



0차원 텐서 (스칼라)

차원이 없는 하나의 숫자 값입니다.

예를 들어 5와 같은 단일 값을 나타냅니다.



2차원 텐서 (행렬)

행과 열, 즉 2개의 축을 가진 숫자들의 격자 배열입니다.

[[1, 2], [3, 4]]와 같은 형태입니다.



1차원 텐서 (벡터)

하나의 축을 따라 나열된 숫자들의 배열입니다.

[1, 2, 3]과 같은 형태로 표현됩니다.



3차원 텐서 이상

3개 이상의 축을 가진 다차원 배열로,

컬러 이미지나 시차원열 데이터 등을 표현하는 데 사용됩니다.

가장 간단한 형태인 스칼라를 만들어보겠습니다.

`torch.tensor()` 함수에 숫자 하나를 넣어 생성할 수 있습니다.

```
# 스칼라 생성
r0 = torch.tensor(1.0).float()
# type 확인
print(type(r0))
# dtype(데이터 타입) 확인
print(r0.dtype)
```

`.float()` 함수를 호출하여 데이터 타입을 `torch.float32`로 명시적으로 지정했습니다.

이는 PyTorch 레이어와의 호환성을 위해 중요합니다.

출력 결과

```
<class 'torch.Tensor'>
torch.float32
```

`shape` 속성은 텐서의 각 차원별 크기를 알려줍니다.
0차원 텐서는 차원이 없으므로 `torch.Size([])`와
같이 비어있는 형태로 출력됩니다.

2차원 텐서 (행렬)

```
# 2차원 넘파이 변수 작성
r2_np = np.array([[1, 5, 6], [4, 3, 2]])
print(r2_np.shape) # >>> (2, 3)

# 넘파이에서 텐서로 변환
r2 = torch.tensor(r2_np).float()

print(r2.shape)
# shape 확인 torch.Size([2, 3])
print(r2.data)
# 데이터 확인 # tensor([[1., 5., 6.], [4., 3., 2.]])
```

2x3 형태의 NumPy 배열이 torch.Size([2, 3]) 형태의 2차원 텐서로 성공적으로 변환되었습니다.

3차원 텐서

```
# 난수 seed 초기화 (결과 재현을 위해)
torch.manual_seed(123)

# shape가 [3, 2, 2]인 정규분포 텐서 작성
r3 = torch.randn((3, 2, 2))

print(r3.shape)
# shape 확인 >>> torch.Size([3, 2, 2])
print(r3.data)
# tensor([[-0.1115,  0.1204], .... ])
```

torch.randn((3, 2, 2))는 평균 0, 분산 1의 정규분포를 따르는 난수로 채워진 3x2x2 텐서를 생성합니다.

view() 함수는 텐서의 형태(shape)를 바꿔주는 역할을 합니다.

NumPy의 reshape 함수와 동일하며, 원소의 총 개수만 유지된다면 원하는 형태로 변경할 수 있습니다.

1

원본 텐서`r3 (shape: [3, 2, 2])``총 원소 개수: 12개`

2

2차원 텐서로 변환

```
r6 = r3.view(3, -1)
print(r6.shape) # [3, 4]
```

3

1차원 텐서로 변환

```
r7 = r3.view(-1)
print(r7.shape) # [12]
```

‘-1’을 지정하면 PyTorch가 총 원소 개수를 맞추기 위해 해당 차원의 크기를 자동으로 차원 계산을 합니다.

이 기능은 코드를 매우 유연하게 만들어 줍니다.

requires_grad

자동 미분 기능과 관련이 있습니다.
True로 설정하면 이 텐서에 대한 모든
연산이 추적되어 그래디언트를 차원산할
수 있게 됩니다.

```
print(r1.requires_grad) # False
```

device

텐서가 어느 장치(CPU 또는 GPU)에
저장되어 있는지를 나타냅니다.
GPU 연산 시 중요한 속성입니다.

```
print(r1.device) # cpu
```

item() 함수

0차원 텐서(스칼라)에서 파이썬의 기본
숫자 값을 추출할 때 사용합니다.
주로 모델의 손실 값을 기록할 때
유용합니다.

```
item_val =  
r0.item()print(type(item_val)) # float
```


텐서 내의 최댓값을 찾는 `max` 함수는 딥러닝에서 매우 중요합니다.

특히 다중 분류 모델의 예측 결과를 해석할 때 핵심적으로 사용됩니다.

전체 최댓값

```
# 텐서 r2 확인
print(r2) # >>> tensor([[1., 5., 6.], [4., 3., 2.]])
# max 함수를 인수 없이 호출하면, 최댓값을 얻음
print(r2.max()) # >>> tensor(6.)
```

인자 없이 호출하면 모든 원소 중 최댓값을 반환합니다.

축별 최댓값

```
# torch.max 함수
# 두 번째 인수는 기준이 되는 축을 의미함 (1은 행 방향)
print(torch.max(r2, 1))

# 몇 번째 요소가 최댓값인지 indices 확인을 통해 알 수 있음
print(torch.max(r2, 1)[1])
# 축을 지정하면 해당 축을 기준으로 최댓값과 그 인덱스를 함께 반환합니다.
```

머신러닝 모델을 학습시킨다는 것은, 모델이 정답을 더 잘 맞히도록 '최적의 파라미터'를 찾아가는 과정입니다.

이 과정은 마치 안개가 자욱한 산에서 가장 낮은 차원곡을 찾아 내려가는 것과 같습니다.

현재 위치에서 어느 방향으로 가야 가장 가파르게 내려갈 수 있을까요?

바로 '경사(Gradient)', 즉 미분값을 차원산하여 현재 위치의 기울기를 알아내야 합니다.

01

requires_grad 사용

`requires_grad=True` 속성을 반드시 설정

03

시각화

`make_dot`과 같은 유틸리티 함수를 사용하면,
PyTorch가 내부적으로 생성한 그래프를 시각적으로 확인 가능.

05

경사값 가져오기

각 텐서의 `.grad` 속성을 통해 해당 텐서의 경사값 확인

02

텐서 변수 간의 연산 수행

`requires_grad=True`로 설정된 텐서들 간에 연산이 수행되면,
PyTorch는 내부적으로 이 모든 연산과정을 그래프로 저장.

04

backward()

`.backward()` 함수를 호출하면, PyTorch는 그래프를 거꾸로 거슬러 올라가며
각 변수에 대한 경사를 자동으로 계산산.

06

경사값 초기화

PyTorch는 `.backward()`를 호출할 때마다 경사값을 기존 값에 누적함
따라서 새로운 경사를 계산하기 전에 `.zero_()` 함수로 초기화해야 합니다.

$y=2x^2+2$ 라는 2차 함수의 기울기를 구하는 과정

```
# x를 넘파이 배열로 정의
x_np = np.arange(-2, 2.1, 0.25)

# x값 표시 >>> [-2.   -1.75 -1.5  -1.25 -1.   -0.75 -0.5  -0.25  0.    0.25  0.5   0.75  1.    1.25  1.5   1.75  2. ]

# requires_grad = True로 설정하여 텐서 생성
x = torch.tensor(x_np, requires_grad=True, dtype=torch.float32)

# 결과 확인 >>> tensor([-2.0000, -1.7500, -1.5000, -1.2500, -1.0000, -0.7500, -0.5000, -0.2500,  0.0000,  0.2500,  0.5000,  0.7500,  1.0000,  1.2500,  1.5000,  1.7500,  2.0000], requires_grad=True)
```

❏ `requires_grad=True` 설정이 가장 중요합니다. 이 설정을 통해 PyTorch에게 x 텐서에 대한 모든 연산을 추적하도록 명령합니다.

정의된 x 텐서를 사용하여 2차 함수를 차원산합니다.

x 가 텐서이므로, 그 결과인 y 역시 자동으로 텐서가 됨.

```
y = 2*x2 + 2
```

```
# 최종 값은 스칼라일 필요가 있으므로, sum 함수를 붙임
```

```
z = y.sum()
```

`.backward()` 함수를 호출하기 위해서는 결과값이 스칼라여야 합니다.

현재 y 는 여러 값을 가진 벡터이므로, `.sum()` 함수를 적용하여 스칼라 값 z 를 만듭니다.

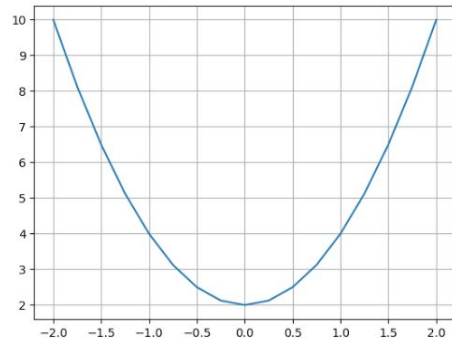
grad_fn 정보

y 의 출력 결과를 확인 시, 값뿐만 아니라

`grad_fn=<AddBackward0>` 라는 정보가

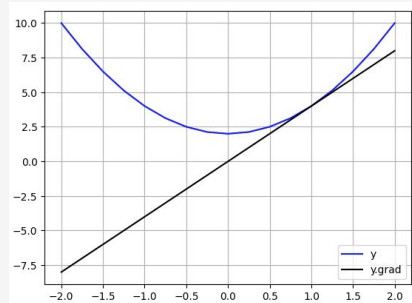
추가된 것을 볼 수 있습니다.

```
tensor([10.0000,  8.1250,  6.5000,  5.1250,  4.0000,  3.1250,  2.5000,  2.1250,
         2.0000,  2.1250,  2.5000,  3.1250,  4.0000,  5.1250,  6.5000,  8.1250,
        10.0000], grad_fn=<AddBackward0>)
```



최종 결과 `z`에 `.backward()` 함수를 호출하기만 하면,
PyTorch가 역으로 미분을 수행하고 `x`의 기울기를 계산

```
z.backward()
# 경사값 가져오기
print(x.grad) # >>> tensor([-8., -7., -6., ..., 6., 7., 8.])
```



$y=2x^2+2$ 를 미분하면 $y'=4x$ 가 됩니다. 출력된 `x.grad` 값은 `x` 값에 정확히 4를 곱한 값과 일치합니다.

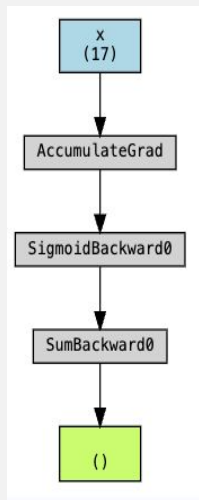
이로써 자동 미분이 정확하게 수행되었음을 확인했습니다.

새로운 연산 수행을 위해서는 반드시 기존 기울기를 0으로 초기화해야 합니다:

```
# 경사값의 초기화
x.grad.zero_()
```

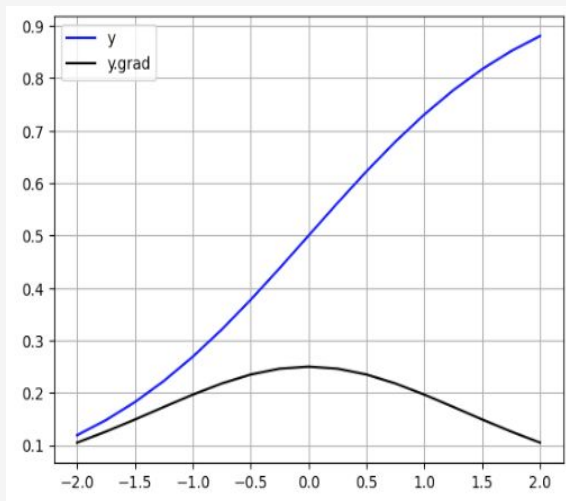
```
sigmoid = torch.nn.Sigmoid() # 시그모이드 함수  
정의
```

```
y = sigmoid(x)  
z = y.sum()  
# 최종 결과를 스칼라로 변환  
z.backward()
```



PyTorch에 내장된 `torch.nn.Sigmoid()` 함수를 사용했습니다.

이전과 달리 복잡한 연산 조합이 아닌 `SigmoidBackward` 라는 단일 노드로 표시됩니다.



딥러닝에서 컬러 이미지를 다룰 때 4차원 텐서를 기본으로 사용하는 이유를 살펴보겠습니다.

텐서 1 & 2(세로, 가로)

이미지는 기본적으로 2차원의 넓이를 가지므로 세로(height), 가로(width) 2개의 텐서가 필요합니다.

텐서 3(채널)

컬러 이미지는 빛의 삼원색인 Red, Green, Blue(RGB) 각 채널마다 2차원 배열이 한 장씩, 총 3장이 겹쳐진 형태입니다.

텐서 4(데이터 개수)

모델을 학습시킬 때는 보통 여러 장의 이미지를 한 묶음(batch)으로 처리합니다. 각 이미지를 구분하기 위한 텐서가 필요합니다.

따라서 학습용 이미지 데이터는 (데이터 개수, 채널, 세로, 가로)의 4차원 텐서로 다루어집니다.

기울기 (Slope)

직선 $y=mx+n$

>> x가 1 증가할 때 y가 변화하는 크기를 의미합니다.

1

경사 (Gradient)

기울기와 미분이 2차원 평면에서의 이야기였다면,
경사는 3차원 이상의 공간으로 확장된 개념입니다.

현재 위치에서 동, 서, 남, 북 어느 방향으로 가야
가장 가파른지를 알려줍니다.

3

2

미분 (Derivative)

직선이 아닌 '곡선'에서 특정 한 점의 '접선의 기울기'를
의미합니다.

기울기 공식에서 두 점의 간격을 0에 가깝게 극한으로 보냈을
때의 값으로, '순간 변화율'이라고도 볼 수 있습니다.

머신러닝에서는 손실 함수(Loss Function)의 값을 최소화하는 것이 목표입니다.

가장 단순한 선형 회귀 모델조차 파라미터가 W와 B, 최소 두 개입니다.

여러 변수(파라미터)를 가진 다변수 함수에서는 각 변수에 대해 개별적으로 미분하는 **편미분(Partial Derivative)**을 해야 합니다.

$$\nabla := \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right)$$

이렇게 각 파라미터로 편미분한 결과들을 모아놓은 벡터(1차원 텐서)가 바로 경사(Gradient)입니다.

따라서 머신러닝에서 손실 함수의 미분을 '경사'라고 부르는 것은 매우 자연스러운 일입니다.

Define-and-run (정적 그래프)

차원산 그래프를 마치 설차원도처럼 먼저 정의(Define)한 뒤, 실행(run) 시점에 데이터를 흘려보내 결과를 얻습니다.

- TensorFlow 1.x 버전이 사용
- 미리 정의된 그래프 구조
- 최적화에 유리하지만 디버깅 어려움

PyTorch는 처음부터 Define-by-run 방식을 채택했으며, 이는 코드가 직관적이고 파이썬의 디버깅 도구를 그대로 사용할 수 있어 훨씬 편리합니다.

TensorFlow 역시 2.0 버전부터 이 방식을 기본으로 채택했습니다.

Define-by-run (동적 그래프)

코드가 실행되는 과정(run)에서 연산 그래프가 동적으로 생성(Define)됩니다.

- PyTorch가 처음부터 채택
- 실행 시점에 그래프 생성
- 직관적이고 디버깅 용이